

Pure Coordinate Transformation Matrix Engine

Problem 3.2 (Dart) · Team 3: Multi-Dimensional Arrays & Functions

Luis Casillas de la Cruz

Jesus Romero Isiordia

Angel Vela Flores

Mobile Programming II · Prof. I.S.C. Luis Gerardo Aguirre Cervantes

The problem: pure 3D transformations

- Robotics kinematics: millions of 3D transformations per second.
- Thread-safety and zero side effects: pure functions only.
- Immutable homogeneous 4×4 matrices (private Float64List).
- Pure functions: Translation, Z-Axis Rotation, Composite Multiplication.
- 3D point transformation with homogeneous normalization ($w = 1.0$).

No external libraries: all the linear algebra is implemented by hand.

Theory: homogeneous coordinates

In 3D, translation is an addition ($p + t$), not a product. Adding a fourth coordinate $w = 1$ turns the point into a 4D vector, so translation becomes a matrix product too. Any chain of transformations collapses into one 4×4 matrix.

3D point



Normalization later divides by w , so the result is always back to 3D.

Translation matrix $T(tx, ty, tz)$

The offset lives in the last column

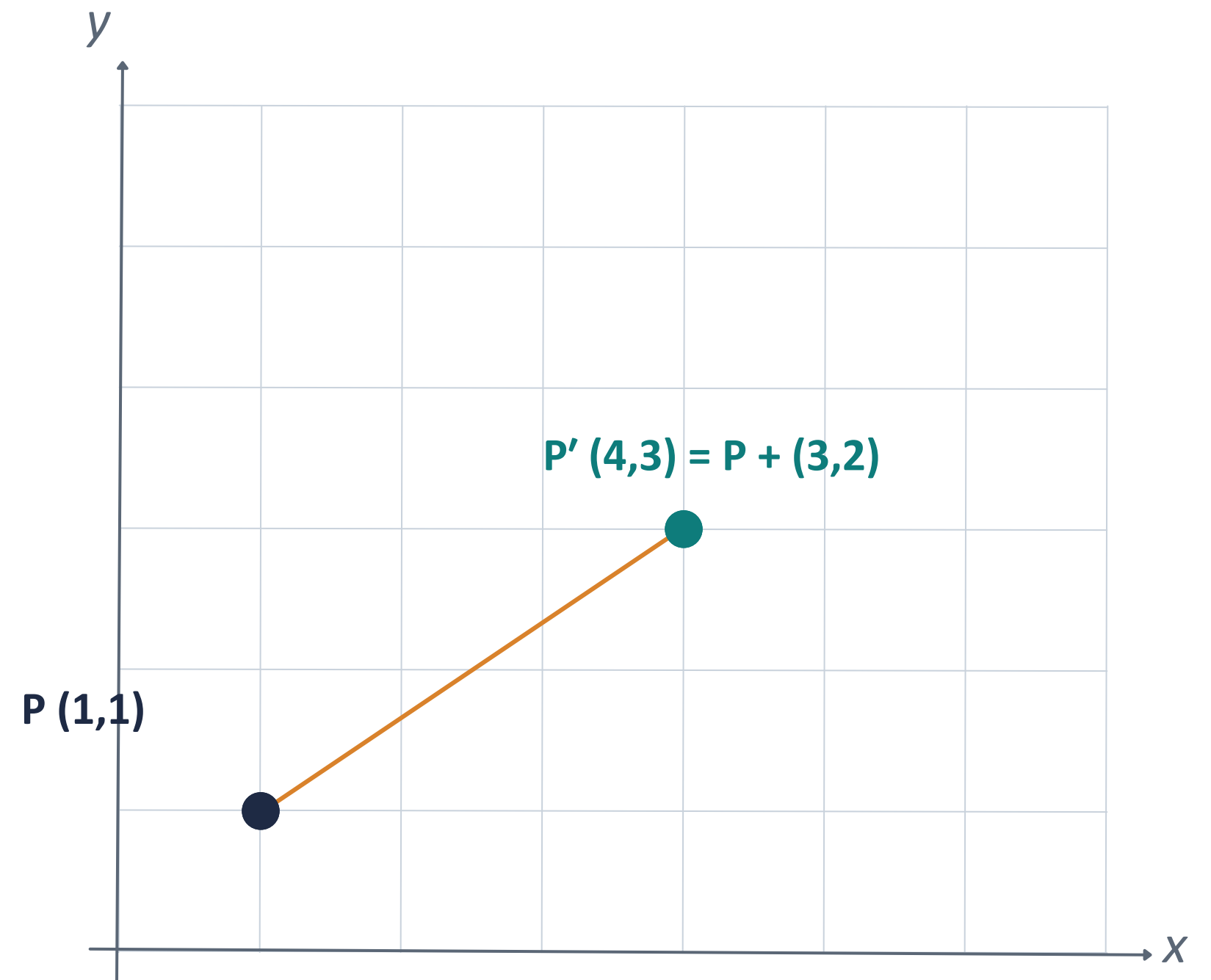
$$\begin{vmatrix} 1 & 0 & 0 & tx \\ 0 & 1 & 0 & ty \\ 0 & 0 & 1 & tz \\ 0 & 0 & 0 & 1 \end{vmatrix}$$

$$x' = x + tx$$

$$y' = y + ty$$

$$z' = z + tz$$

Shape and orientation do not change;
every point slides by the same vector.



Rotation matrix $R_z(\theta)$

Rotates the xy-plane, z stays fixed

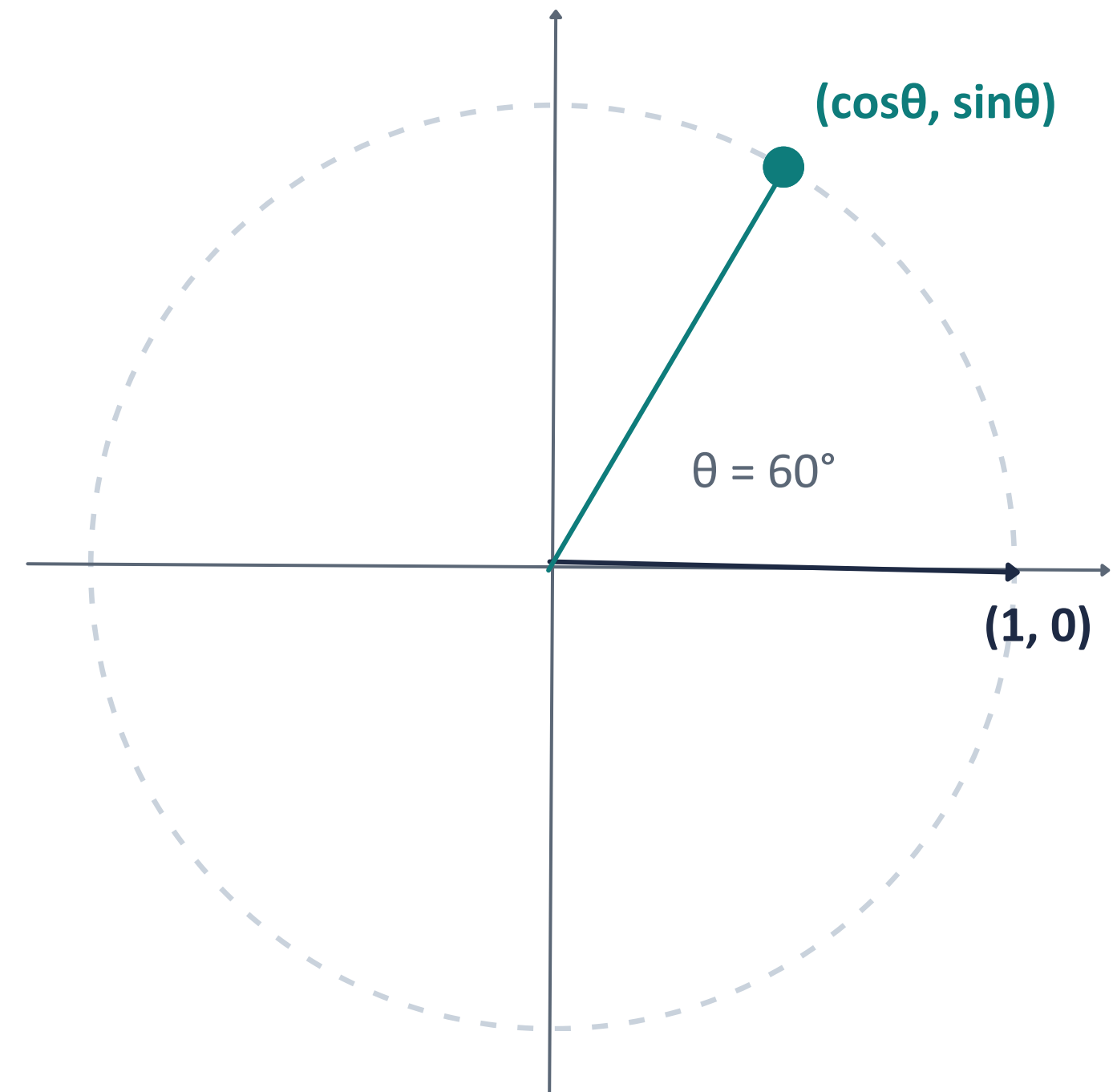
$$\begin{vmatrix} \cos\theta & -\sin\theta & 0 & 0 \\ \sin\theta & \cos\theta & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{vmatrix}$$

$$x' = x \cdot \cos\theta - y \cdot \sin\theta$$

$$y' = x \cdot \sin\theta + y \cdot \cos\theta$$

$$z' = z$$

The columns are where the x and y axes land after turning by θ .



How a point is transformed, step by step

Each output coordinate is one matrix row $\times$ the vector. Example: $T(5,2,1)$ applied to $P = (1,1,1)$.

$$\begin{array}{|c|c|c|c|c|} \hline 1 & 0 & 0 & 5 & \\ \hline 0 & 1 & 0 & 2 & \\ \hline 0 & 0 & 1 & 1 & \\ \hline 0 & 0 & 0 & 1 & \\ \hline \end{array}$$

$\times$

$$\begin{array}{|c|} \hline 1 \\ \hline 1 \\ \hline 1 \\ \hline 1 \\ \hline \end{array}$$

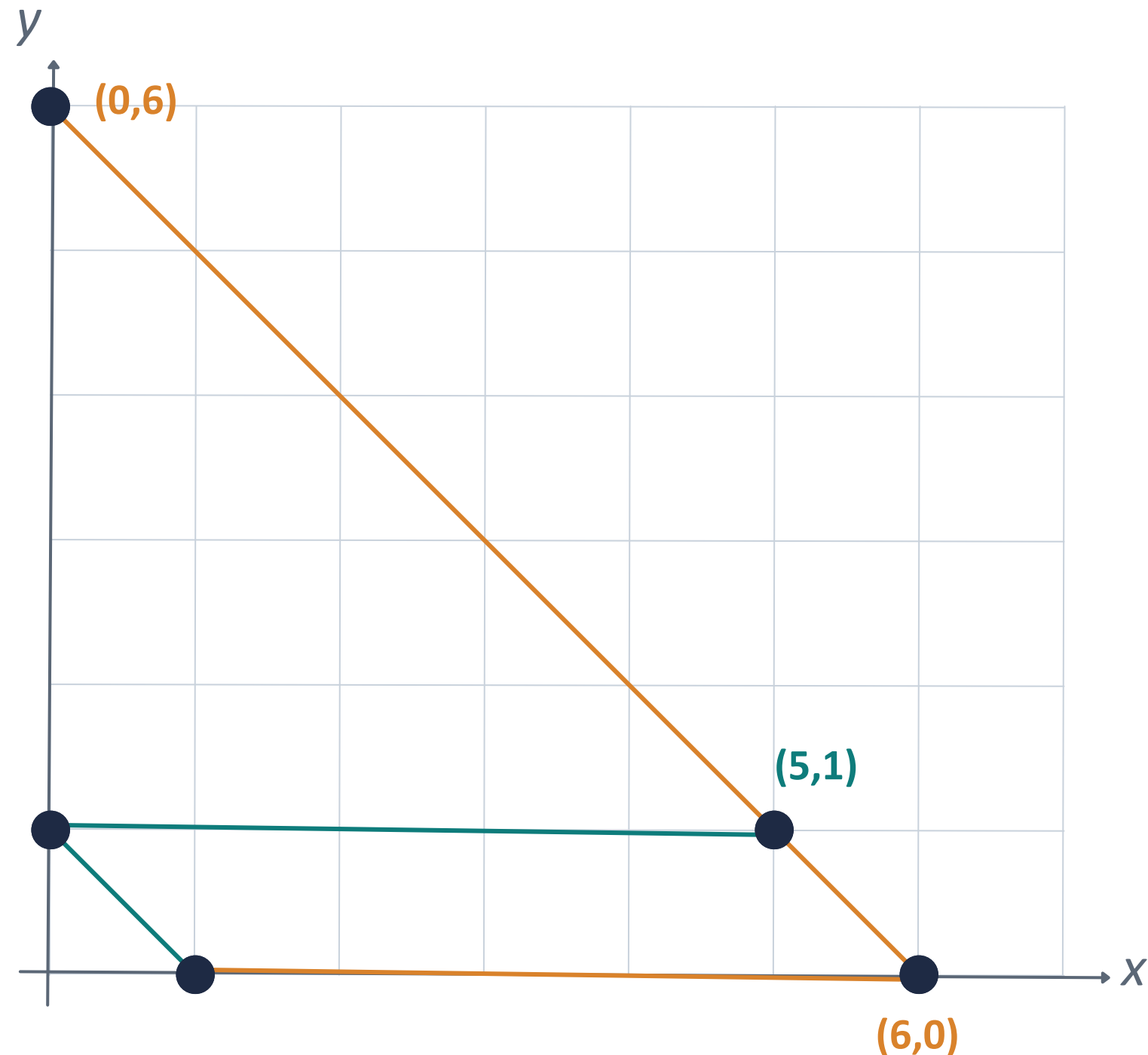
$=$

$$\begin{array}{|c|} \hline 6 \\ \hline 3 \\ \hline 2 \\ \hline 1 \\ \hline \end{array}$$

Row 1:
 $1 \cdot 1 + 0 \cdot 1 + 0 \cdot 1 + 5 \cdot 1 = 6$
Row 2:
 $0 + 1 + 0 + 2 = 3$
Row 3:
 $0 + 0 + 1 + 1 = 2$

Run `transform_engine.dart`: the terminal prints every matrix and this same row-by-row trace.

Composition: order matters



Apply A, then B, then C:

$$M = C \cdot B \cdot A$$

Rotate 90°, then translate (5,0)

$$M = T \cdot R_z \quad (1,0) \rightarrow (0,1) \rightarrow (5,1)$$

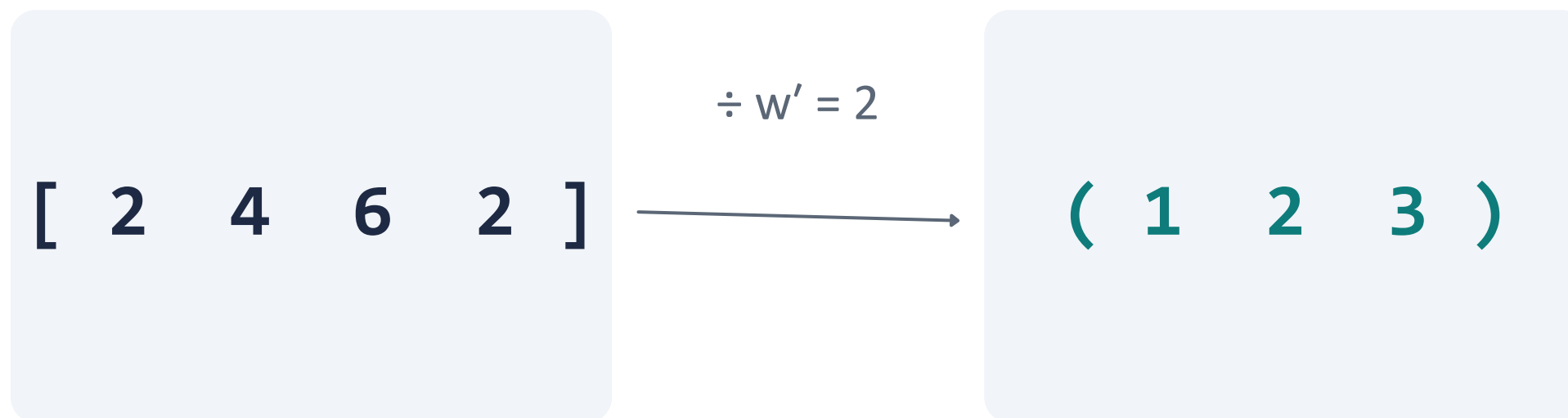
Translate (5,0), then rotate 90°

$$M = R_z \cdot T \quad (1,0) \rightarrow (6,0) \rightarrow (0,6)$$

Matrix products are associative but not commutative.

Homogeneous normalization (w)

After $M \cdot [x \ y \ z \ 1]^T$ we get $[x' \ y' \ z' \ w']^T$. The Cartesian point is $(x'/w', y'/w', z'/w')$.



Affine matrices (T, Rz) give $w' = 1$: no division.

$w' = z$: simple perspective, $(2,4,2) \rightarrow (1,2,1)$.

$w' \approx 0$: point at infinity, the engine throws `StateError` instead of returning ∞ or NaN.

Guard: $|w'| < 1e-12$ is treated as zero.

Code logic

- Matrix4: final class, private Float64List, defensive copy in fromRows.
- Validation: 4×4 shape, finite values, valid indices.
- Stateless top-level functions: translation, rotationZ, multiply, compose, transformPoint.
- Each call returns a new instance, so it is safe across isolates.

```
Matrix4 multiply(Matrix4 a, Matrix4 b)
{
    final d = Float64List(16);
    for (i in 0..3)
        for (j in 0..3) {
            sum = 0.0;
            for (k in 0..3)
                sum += a[i][k] * b[k][j];
            d[i*4+j] = sum;
        }
    return Matrix4._(d);
}
```

Complexity and verification

Operation	Time	Space
translation / rotationZ	$O(1)$	$O(1)$
multiply (4×4)	$O(n^3) = 64$ products	$O(1)$
transformPoint	$O(n^2) = 16$ products	$O(1)$
compose (k matrices)	$O(k)$	$O(1)$ extra

- Tests in main(): identity, T, Rz, composition, associativity, $R(\theta) \cdot R(-\theta) = I$.
- Edge cases: $w = 0$, NaN, ∞ , 3×3 matrix, bad index, 1e6 rad.
- Purity: inputs unchanged, defensive copy.
- Benchmark: 2 million transformations.

A dark blue L-shaped graphic element consisting of a vertical bar on the left and a horizontal bar extending to the right, positioned behind the text.

**Thank You For Your
Attention**